

Sesión 10: Rotaciones Dobles en Árboles AVL

Estrategias de Balanceo Compuesto, Resolución de Desviaciones en Zig-Zag y Verificación de Estabilidad en Escenarios Masivos

IPC 2 | Facultad de Ingeniería USAC

Laboratorio - Primer Semestre 2026

| Agenda del Día (1/2)

- ✂ El Desbalanceo Cruzado o en Doble Sentido
- 🔗 Anatomía de las Rotaciones Compuestas
- ↻ Rotación Doble Izquierda-Derecha (RID)
- ↻ Rotación Doble Derecha-Izquierda (RDI)



Estabilidad Total

Completaremos el ciclo de vida del árbol AVL, permitiendo soportar cualquier patrón de inserción aleatoria sin perder eficiencia.

Agenda del Día (2/2)



Pruebas Masivas

Verificación del algoritmo procesando colecciones aleatorias complejas.



Refactorización Inserción

Integración final de los 4 casos de desbalanceo en la función recursiva C#.



Responsabilidad

Valor de la unidad: Construcción de algoritmos robustos y tolerantes a la carga de datos.

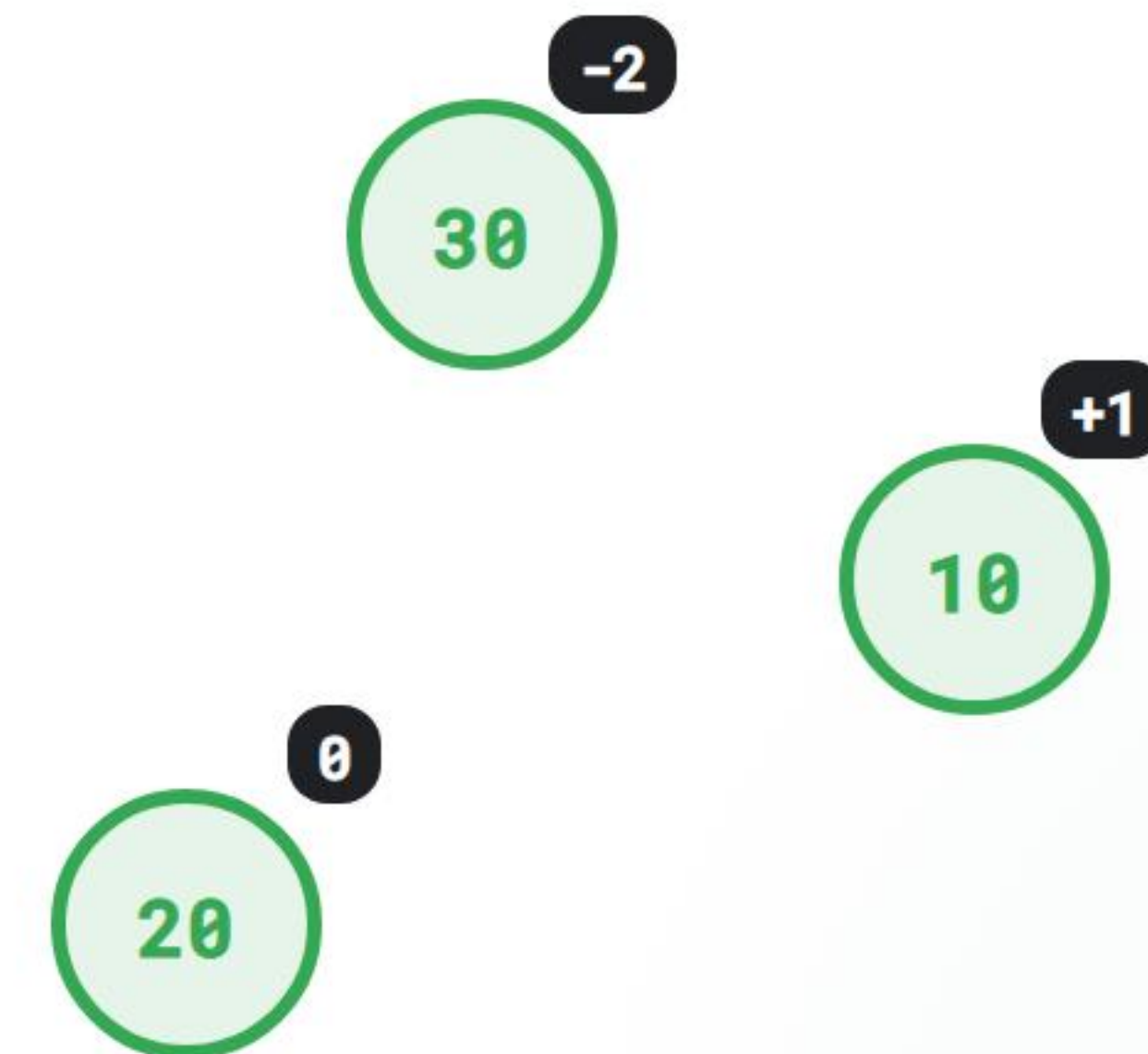
El Límite de las Rotaciones Simples

Por qué el desbalanceo en forma de "Zig-Zag" requiere una estrategia compuesta

Cuando la Rotación Simple Falla

Las rotaciones simples (RLL y RRD) asumen que la desviación del árbol ocurre en una línea recta continua. Pero, ¿qué pasa si el desbalanceo se produce en forma cruzada?

El Caso Complejo: Si insertamos los valores 30, 10 y luego 20, el árbol se inclina a la izquierda, pero el último nodo cuelga a la derecha. Aplicar una única rotación simple aquí **no resuelve el problema**; solo cambia la inclinación de lado.



¿Qué es una Rotación Doble?

Una rotación doble no es un algoritmo nuevo e independiente. Es la **combinación secuencial y coordinada de dos rotaciones simples** aplicadas a diferentes niveles jerárquicos de un subárbol.

Principio de Funcionamiento:

1. La **primera rotación** se ejecuta sobre el hijo del nodo desbalanceado, alineando el subárbol para transformarlo en un caso lineal recto.
2. La **segunda rotación** se aplica directamente sobre el nodo padre original, logrando el equilibrio perfecto.

Rotación Doble Izquierda-Derecha (RID)

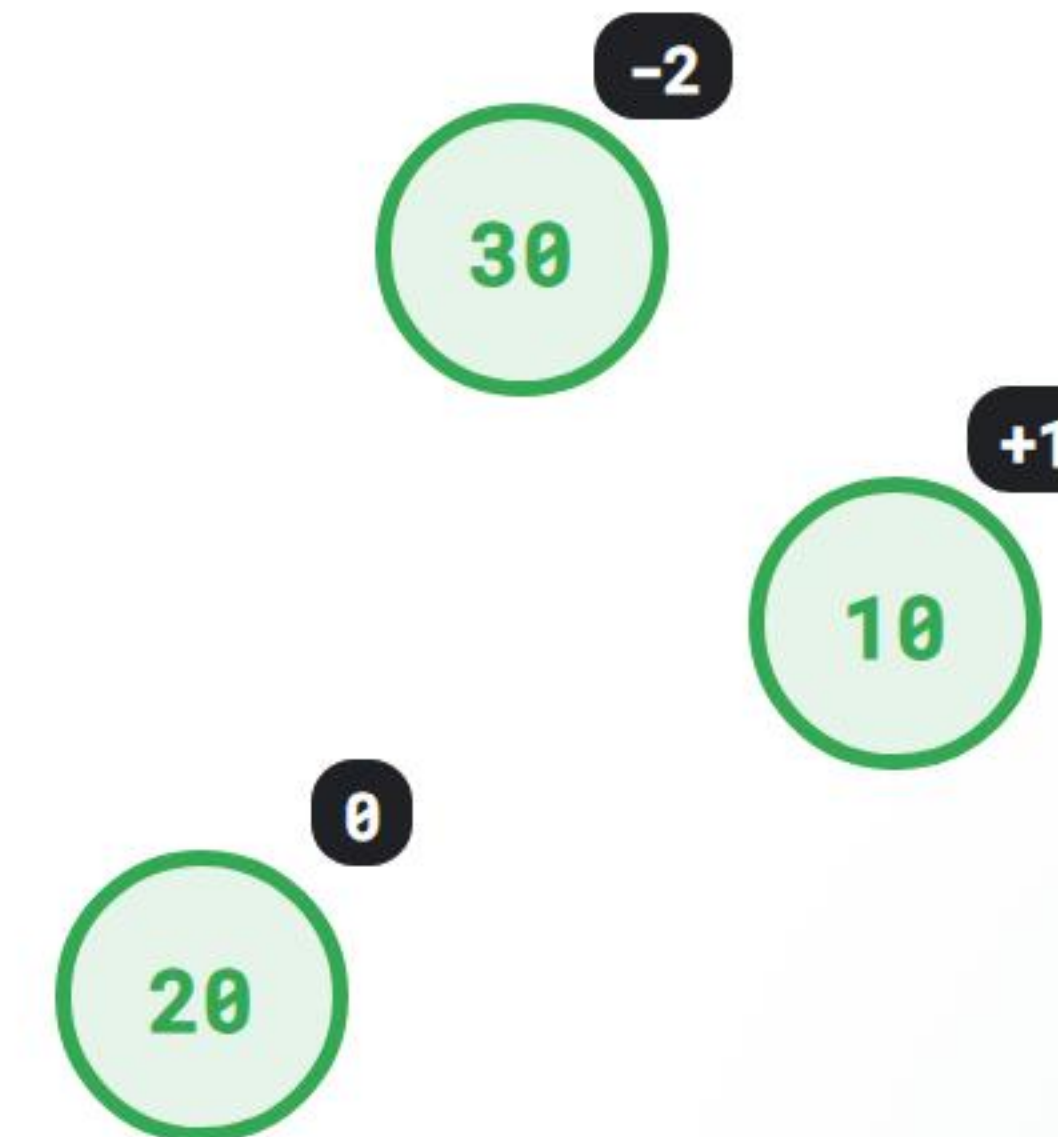
Resolución del desbalanceo interno Izquierda-Derecha (Hijo izquierdo, Nieto derecho)

Rotación Izquierda-Derecha (RID): El Origen

Este caso ocurre cuando un nodo tiene un **FE de -2** y su hijo izquierdo tiene un **FE de +1**.

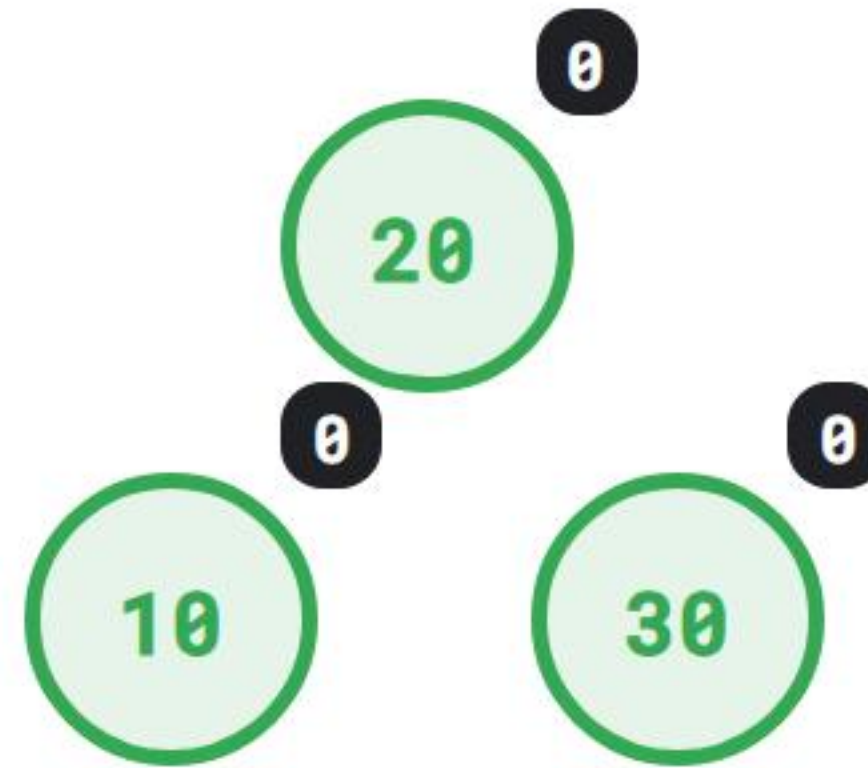
Mecánica del Proceso:

1. Aplicamos primero una **Rotación Simple a la Izquierda** sobre el hijo izquierdo.
2. Esto convierte la rama en un desbalanceo lineal Izquierda-Izquierda puro.
3. Finalmente, ejecutamos una **Rotación Simple a la Derecha** sobre el abuelo o nodo raíz original.



Efecto de la Rotación RID en Memoria

Observemos el estado de simetría óptimo tras aplicar de forma consecutiva ambas transformaciones de punteros:



✓ El "Zig-Zag" ha sido neutralizado. Alturas y factores sincronizados en cero.

Código C#: Implementación de la Rotación RID

Gracias a la reutilización limpia de componentes y funciones de la clase anterior, el código es directo y elegante:

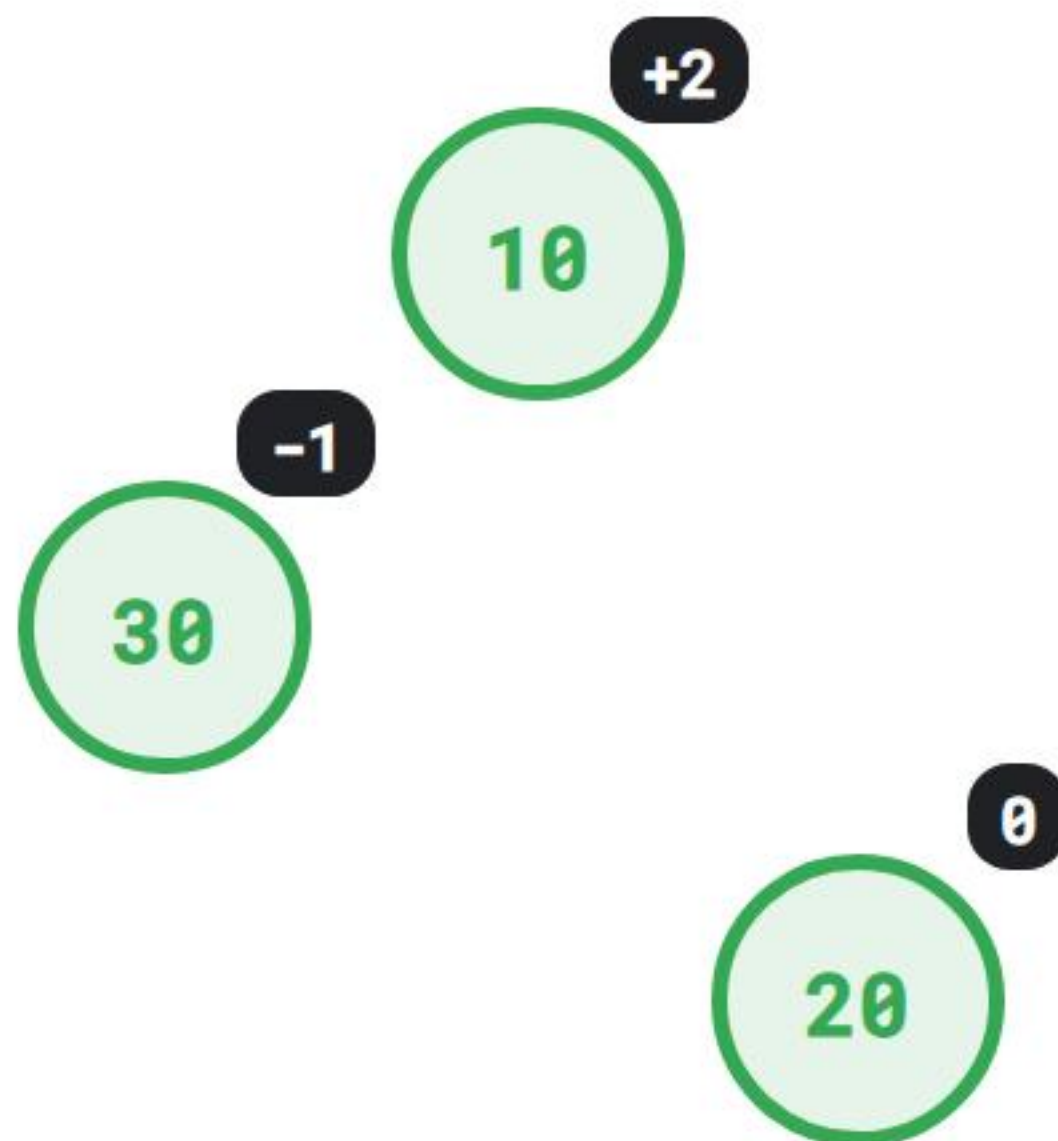
```
private NodoAVL RotarIzquierdaDerecha(NodoAVL nodo) {  
    // Paso 1: Alinear el subárbol inferior rotando a la izquierda  
    nodo.Izquierdo = RotarIzquierda(nodo.Izquierdo);  
  
    // Paso 2: Resolver el desbalanceo lineal rotando a la derecha  
    return RotarDerecha(nodo);  
}
```

Rotación Doble Derecha-Izquierda (RDI)

Resolución del desbalanceo interno Derecha-Izquierda (Hijo derecho, Nieto izquierdo)

Rotación Derecha-Izquierda (RDI): El Origen

Este caso se presenta de forma simétrica inversa: cuando un nodo padre presenta un **FE de +2** y su hijo derecho directo posee un **FE de -1**.

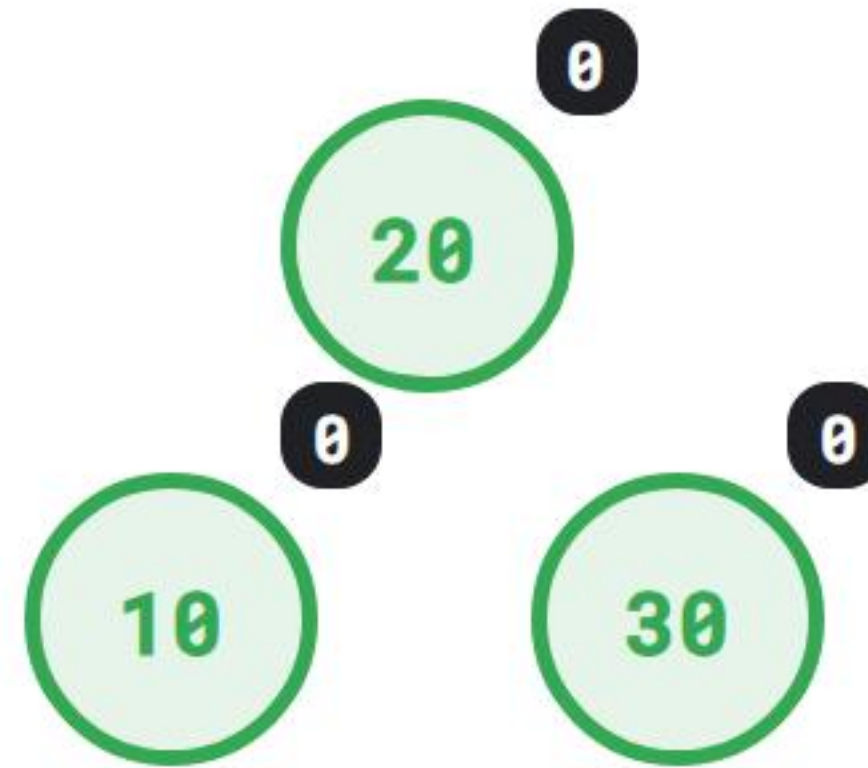


Mecánica del Proceso:

1. Ejecutamos primero una **Rotación Simple a la Derecha** sobre el hijo derecho.
2. La estructura se re-alinea, convirtiéndose en un desbalanceo Derecho-Derecho lineal.
3. Completamos la operación ejecutando una **Rotación Simple a la Izquierda** sobre la raíz local.

Efecto de la Rotación RDI en Memoria

Visualicemos el resultado de la reestructuración física de las referencias en memoria:



- ✓ El árbol recupera el orden y balance ideal de forma completamente automatizada.

Código C#: Implementación de la Rotación RDI

Nuevamente orquestamos de forma modular aprovechando las primitivas de rotación ya programadas:

```
private NodoAVL RotarDerechaIzquierda(NodoAVL nodo) {  
    // Paso 1: Alinear el subárbol inferior rotando a la derecha  
    nodo.Derecho = RotarDerecha(nodo.Derecho);  
  
    // Paso 2: Resolver el desbalanceo lineal rotando a la izquierda  
    return RotarIzquierda(nodo);  
}
```

Integración y Refactorización del Motor AVL

Unificación de los 4 escenarios de balanceo en la rutina de inserción principal

El Método de Inserción Completo (1/3)

```
private NodoAVL InsertarRecursivo(NodoAVL actual, int valor) {
    if (actual == null) return new NodoAVL(val);

    // 1. Descenso binario tradicional
    if (valor < actual.Dato) {
        actual.Izquierdo = InsertarRecursivo(actual.Izquierdo, valor);
    } else if (valor > actual.Dato) {
        actual.Derecho = InsertarRecursivo(actual.Derecho, valor);
    } else return actual; // Llave duplicada

    // 2. Actualización de la altura local
    actual.Altura = 1 + Math.Max(ObtenerAltura(actual.Izquierdo), ObtenerAltura(actual.Derecho));

    // 3. Obtener Factor de Equilibrio
    int fe = ObtenerFactorEquilibrio(actual);
    // (Continúa en la siguiente slide...)
```


El Método de Inserción Completo (2/3)

```
// --- CONTROL DE DESBALANCEOS IZQUIERDOS (fe = -2) ---  
if (fe < -1) {  
    // Caso Izquierda-Izquierda (Lineal) -> Rotación Simple  
    if (valor < actual.Izquierdo.Dato) {  
        return RotarDerecha(actual);  
    }  
    // Caso Izquierda-Derecha (Cruzado) -> Rotación Doble  
    if (valor > actual.Izquierdo.Dato) {  
        return RotarIzquierdaDerecha(actual);  
    }  
}  
// (Continúa en la siguiente slide...)
```


El Método de Inserción Completo (3/3)

```
// --- CONTROL DE DESBALANCEOS DERECHOS (fe = +2) ---
if (fe > 1) {
    // Caso Derecho-Derecho (Lineal) -> Rotación Simple
    if (valor > actual.Derecho.Dato) {
        return RotarIzquierda(actual);
    }
    // Caso Derecho-Izquierda (Cruzado) -> Rotación Doble
    if (valor < actual.Derecho.Dato) {
        return RotarDerechaIzquierda(actual);
    }
}

return actual; // El nodo no sufrió desbalanceo, se retorna intacto
}
```

Verificación de Auto-Balanceo Masivo

Estrategias de QA y pruebas bajo estrés con colecciones masivas de datos en .NET

¿Por qué probar con Operaciones Masivas?

Validar un árbol AVL únicamente con 3 o 5 datos insertados manualmente no es suficiente para certificar un software empresarial.

Las operaciones masivas nos permiten asegurar que:

- Las rotaciones ****no rompen la propiedad de ordenación**** del ABB subyacente.
- El árbol maneja adecuadamente ****múltiples balanceos en cascada**** en una sola inserción profunda.
- No existen fugas de memoria o ****referencias circulares**** ocultas en los punteros de los subárboles.

Código: Orquestación de Pruebas Masivas

```
public static void Main(string[] args) {
    ArbolAVL arbolMasivo = new ArbolAVL();
    Random constructorAleatorio = new Random();

    Console.WriteLine("Insertando 50,000 llaves aleatorias en el AVL...");
    for (int i = 0; i < 50000; i++) {
        int numero = constructorAleatorio.Next(1, 1000000);
        arbolMasivo.Insertar(numero);
    }

    // Método de control de calidad interno para verificar la altura máxima
    bool esValido = arbolMasivo.VerificarIntegridadAVL();
    Console.WriteLine($"¿Estado estructural del Árbol AVL Válido?: {esValido}");
}
```


Código: Auditoría de Integridad Estructural

Un buen desarrollador programa rutinas para auto-evaluar la calidad de sus estructuras de datos en producción:

```
public bool VerificarIntegridadAVL() { return ValidarNodo(raiz); }

private bool ValidarNodo(NodoAVL nodo) {
    if (nodo == null) return true;

    int fe = ObtenerFactorEquilibrio(nodo);
    if (fe < -1 || fe > 1) return false; // ¡Propiedad AVL rota!

    // Verificación en cascada recursiva para el resto del árbol
    return ValidarNodo(nodo.Izquierdo) && ValidarNodo(nodo.Derecho);
}
```

Complejidad Asintótica Comparativa

Análisis matemático del impacto real del auto-balanceo frente a las estructuras tradicionales

Análisis de Rendimiento: ABB vs AVL

Evaluación en notación Big O de operaciones fundamentales bajo el peor escenario posible:

Operación Básica	ABB Tradicional (Peor Caso)	Árbol AVL (Peor Caso)	Impacto Real
Búsqueda (Search)	$O(n)$ (Inaceptable)	$O(\log n)$ (Óptimo)	Velocidad exponencialmente superior.
Inserción (Insert)	$O(n)$	$O(\log n)$	Incluye el tiempo de las rotaciones.
Eliminación (Delete)	$O(n)$	$O(\log n)$	Mantiene la simetría tras limpiar nodos.

El Costo de Rotar: ¿Afecta la Complejidad?

Una duda recurrente en ciencia de la computación es si el cálculo de alturas y el reajuste físico de punteros durante las rotaciones ralentiza la inserción.

Análisis Matemático Finito: Reasignar punteros de memoria (ej. `nodo.Izquierdo = subArbol;`) representa una operación directa de tiempo constante **$O(1)$** . No requiere bucles ni iteraciones internas. Por lo tanto, el proceso de balanceo mantiene intacta la promesa de eficiencia logarítmica de la estructura.

Buenas Prácticas e Ingeniería de Software

Principios arquitectónicos y de diseño limpio aplicados a estructuras autorreferenciadas

Principio DRY: No repitas código

El peor error metodológico al programar rotaciones dobles es copiar y pegar toda la lógica de reasignación manual de punteros dentro de una función nueva.

Inconveniente Técnico: Escribir asignaciones directas masivas incrementa drásticamente la probabilidad de generar referencias muertas o nulas complejas de depurar en el depurador (Debugger) de Visual Studio.

Enfoque de Ingeniería Profesional: Las rotaciones dobles se componen invocando de manera coordinada las dos funciones simples preexistentes. Esto minimiza errores de desarrollo.

Valor de la Unidad: Responsabilidad



"No se puede escapar de la responsabilidad del mañana evadiéndola hoy."

— Abraham Lincoln

****Aplicación Práctica:**** Desarrollar código tolerante a fallas y óptimo mediante el uso de algoritmos auto-balanceables es una muestra de responsabilidad en ingeniería de software, garantizando sistemas estables ante cargas masivas impredecibles.

Conclusiones de la Sesión

Casos Cruzados

Los desbalanceos en forma de zig-zag no pueden solucionarse mediante rotaciones simples. Requieren reestructuraciones compuestas.

Modularidad Total

Las transformaciones RID y RDI se implementan invocando consecutivamente las operaciones simples preexistentes.

Estabilidad Óptima

El árbol AVL completado asegura un rendimiento $O(\log n)$ en búsquedas, sin importar el orden de inserción de las llaves.

Fin de la Unidad 2: Próxima Sesión

Unidad 3: Arquitecturas Web y Patrón de Diseño MVC

[cite_start]

En la siguiente sesión daremos el salto hacia el desarrollo de aplicaciones empresariales. Estudiaremos la **Sesión 11: Modelado base y arquitecturas de despliegue**, conociendo los esquemas Cliente-Servidor y el patrón N-Capas[cite: 56, 57].

¿Dudas sobre Rotaciones Dobles o Árboles AVL?

Es momento de integrar y validar de forma definitiva sus estructuras de datos.



Foros en UEDI



Soporte: 3021894750101@ingenieria.usac.edu.gt